

Week 2 Day 3 — LangGraph: Stateful, Multi-Step & Cyclical Agent Workflows

Workflow: a Research Assistant agent — plan → retrieve → generate → critique → (revise loop) → human_review → publish/reject — with a bounded self-correction loop, a human-approval gate before the risky action, and full persistence/replay via a checkpointer.

Task 1 — Graph Concepts & State Design

LangGraph's core primitives, and how they map onto this project:

- **StateGraph** is the builder object. You register nodes and edges on it, then `.compile()` it into a runnable graph. It's generic over a state schema, so it knows how to merge each node's return value into the shared state without you writing merge logic by hand.
- **Nodes** are plain functions `def node(state) -> dict`. A node only returns the keys it changed (a *partial update*); LangGraph merges that into the full state before the next node runs. This keeps nodes small and testable in isolation — each one is just “given this slice of state, produce this slice of state.”
- **Edges** are unconditional transitions (`add_edge("a", "b")` — always go from a to b).
- **Conditional edges** are the interesting piece: `add_conditional_edges("a", router_fn, {"x": "b", "y": "c"})`. `router_fn` inspects the state and returns a label; LangGraph follows whichever edge matches. Crucially, nothing stops that label from pointing back at a node that already ran — that's the entire mechanism behind cycles.
- **State** is one TypedDict (or Pydantic model) shared by every node. It's the *only* communication channel between nodes, which is what makes a run inspectable, resumable, and replayable: at any point, “the state” is the complete memory of the run.

State schema (AgentState):

```
class AgentState(TypedDict):
    query: str          # the user's research question
    plan: str           # research plan produced once, up
front
    search_results: List[str] # retrieved snippets (Day 2
search tool)
    draft: str          # current draft answer
    critique: str        # feedback text from the critique
node
    quality_score: float # 0-1 score assigned by the
critique node
    revision_count: int  # how many times we've looped
back to generate
    max_revisions: int   # safety cap on the self-
correction loop
    approved: Optional[bool] # human decision at the interrupt
point
    final_answer: str    # the answer that got published
    log: List[str]       # human-readable trace of every
node that ran
```

Graph sketched before coding (final version, after Tasks 2-5):

Task 4 — Human-in-the-Loop & Interrupts

Treated **publishing** the final answer as the risky action (standing in for “send the email” / “place the order” in a real product). Compiled the graph with `interrupt_before=["publish", "reject"]`, so execution pauses right at that boundary and waits for a human decision instead of just running through.

Demonstrated resuming the same paused run twice, on two separate threads:

- **Rejection:** `update_state(thread, {"approved": False})` then `invoke(None, ...)` to resume → routed to reject, final answer marked *not published*.
- **Approval:** same pattern with `{"approved": True}` → routed to publish, final answer marked *published* with the draft content.

When should a real product require human-in-the-loop

vs. allow full autonomy? Require a human checkpoint when an action is hard to undo, costly if wrong, or reaches outside the system’s own sandbox — sending real emails, spending money, deleting data, publishing under the company’s name — because the downside of a bad autonomous call outweighs the friction of a pause. Full autonomy is reasonable when actions are cheap, reversible, and internal — drafting, searching, calculating, writing to a scratch workspace the agent already owns — where mistakes are low-cost and a self-correction loop like Task 3’s critique cycle is already enough of a safety net on its own.

Task 5 — Persistence & Debugging

Compiled with a `MemorySaver` checkpointer, keyed by `thread_id`. This gives two things for free:

1. **Persistence across sessions** — re-opening a finished thread later (`get_state(thread)`) returns its final state without re-running anything, simulating a process/kernel restart.
2. **Time travel** — `get_state_history(thread)` returns every checkpoint in order. Grabbing an earlier snapshot’s config and calling `invoke(None, config=snapshot.config)` replays the run forward from that exact point — used here to replay from right after retrieve.

AgentExecutor vs. LangGraph — when to reach for each

	LangChain AgentExecutor	LangGraph
Control flow	Implicit, LLM-driven think/act/observe loop	Explicit graph of nodes/edges you design
Branching & cycles	Awkward — limited to what the model’s own reasoning produces	First-class via conditional edges, including loops
Pausing / human approval	Not built in; bolted on manually	Native via <code>interrupt_before</code> / <code>interrupt_after</code>
Persistence & replay	Not built in	Native via checkpointers + state history
Best for	Quick single-shot tool-using agents, prototypes, simple	Multi-step workflows that branch, self-correct, need approval gates, or

In practice: reach for AgentExecutor when the task is “call some tools until you have an answer” and you don’t need to control *how* it loops. Reach for LangGraph as soon as the workflow has more than one logical stage, needs a safety loop, needs a human checkpoint before a risky step, or needs to be pausable/resumable/debuggable as a longer-running process.

An implementation detail worth flagging

The mock LLM and the real Gemini API produced meaningfully different behavior at the critique node, and it’s worth being explicit about why. The mock scores drafts as 0.55 → 0.75 → 0.95 across revisions by construction, so the self-correction loop is *guaranteed* to fire twice before finishing — good for demonstrating the loop mechanics deterministically. Real Gemini, run against the same graph, tended to write a solid draft on the first pass and score it 0.9 immediately, so `route_after_critique` sent it straight to `human_review` without ever visiting `revise`. Neither run is “wrong” — the loop’s *guard logic* is fully exercised either way — but only the mock run actually demonstrates the loop *firing*. That’s a small but real gap between “the code path exists and is correct” and “the code path was observed to execute,” and it’s the kind of thing that’s easy to miss if you only test once against a strong real model. Two ways to close it for a more convincing demo: raise `QUALITY_THRESHOLD` (e.g. to 0.95) so even a good real draft has to revise once, or keep the threshold as-is and simply note in the write-up (as done here) that the loop was verified against the deterministic mock and the guard logic was inspected, but not observed firing on live Gemini output at the default threshold.

Summary

- **Task 1:** defined `StateGraph/nodes/edges/conditional edges/state`, designed the `AgentState` schema, and sketched the target graph before writing any code.
- **Task 2:** built and ran a linear plan → retrieve → generate → format graph, printing state after every node.
- **Task 3:** turned generate → critique into a bounded self-correction loop via a conditional edge and a `revision_count` guard, and explained why this is natural in `LangGraph` but awkward in a plain `AgentExecutor`.
- **Task 4:** added an `interrupt_before` checkpoint in front of the risky `publish` action and demonstrated resuming after both a simulated rejection and a simulated approval, on separate threads.
- **Task 5:** used the `MemorySaver` checkpointer to persist and re-open a thread’s state, and used `get_state_history` to time-travel/replay a run from an earlier checkpoint.